《算法分析与设计》
课后作业

作业编号 作业 7

学　　号 _____

姓　　名 _____

专　　业 _____

学　　院 _____

二 0 二五年六月

第 7 章 回溯法和分支限界法

题目 1（回溯法）

数独游戏是在 9*9 的方格中填放 1~9 的数字，要求每一行、每一列以及 3*3 的方格中的数字均不能相同，如下图所示。

1	4	3	6	2	8	5	7	9
5	7	2	1	3	9	4	6	8
9	8	6	7	5	4	2	3	1
3	9	1	5	4	2	7	8	6
4	6	8	9	1	7	3	5	2
7	2	5	8	6	3	9	1	4
2	3	7	4	8	1	6	9	5
6	1	9	2	7	5	8	4	3
8	5	4	3	9	6	1	2	7

现方格中某些数字为空缺（用 0 表示），希望你编写程序能够将空缺的数字补齐。

输入要求：

输入包含 9 行，每一行包含 9 个数字，对应每个方格中的数字，0 表示该方格的数字为空。

输出要求：

输出为 9 行，每行 9 个数字，也就是补齐后的 9*9 的方格中的数字。

样例输入：

103000509
002109400
000704000
300502006
060000050
700803004
000401000
009205800
804000107

样例输出：

143628579
572139468
986754231
391542786

468917352
725863914
237481695
619275843
854396127

要求：

1、写出问题的目标函数和约束函数。

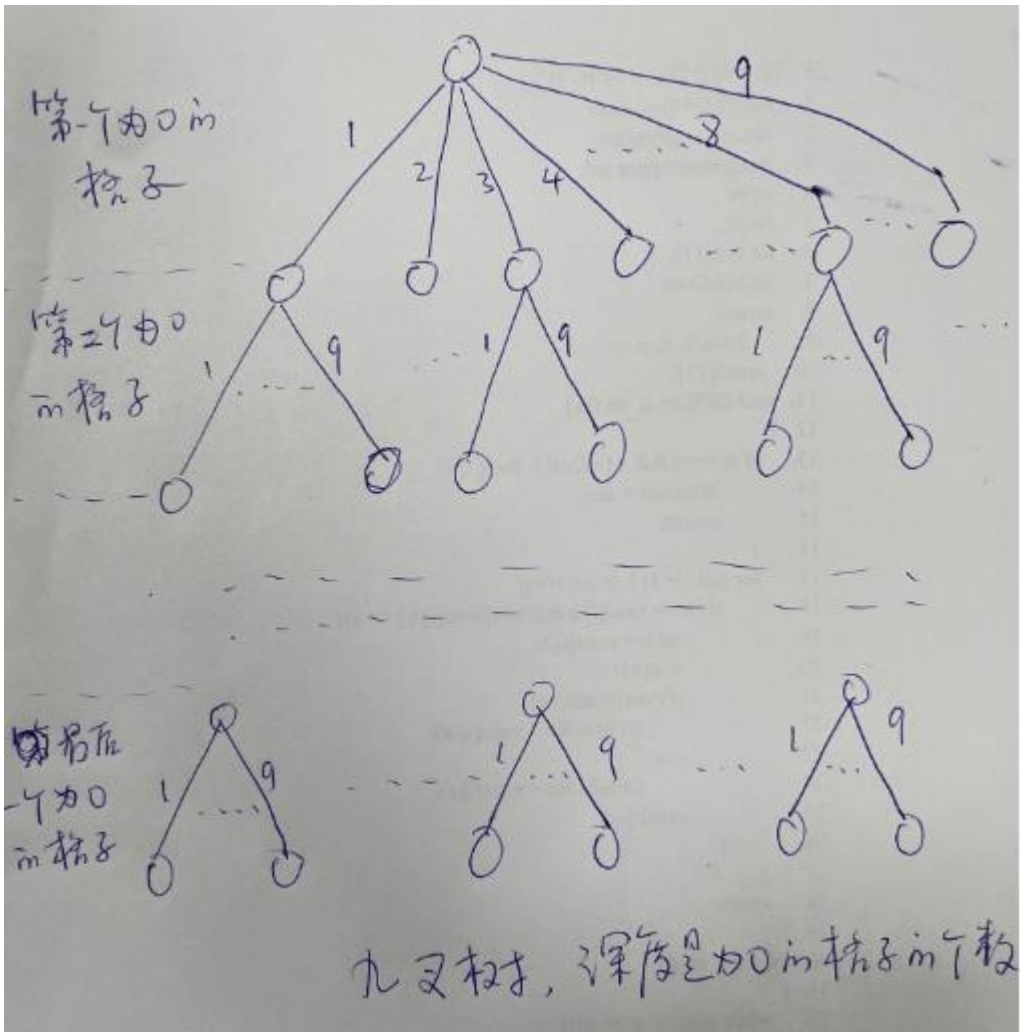
答：

目标函数：按照规则将方格中为 0 的数字替换为 1-9 的某个数字。

约束函数：每一行、每一列以及 3*3 的方格中的数字均不能相同。

2、画出样例输入时的解空间树（标出解空间树的宽度和深度等于什么。如果树的规模太大，可以用省略号）。

答：解空间树为 9 叉树，树的深度为等于 0 的方格的数量。



时间复杂性：9^k（k 等于 0 的方格的数量）

3、参考代码：

```
class Solution {
private:
    bool line[9][9];
```

```
        bool column[9][9];
        bool block[3][3][9];
        bool valid;
        vector<pair<int, int>> spaces;    //棋盘中空白格子的坐标集合
    public:
        void dfs(vector<vector<char>>& board, int pos) {
            if (pos == spaces.size()) {
                valid = true; //所有空白格已经填满
                return;
            }
            auto [i, j] = spaces[pos];
            for (int digit = 0; digit < 9 && !valid; ++digit) {
                if (!line[i][digit] && !column[j][digit] && !block[i / 3][j / 3][digit]) {
                    line[i][digit] = column[j][digit] = block[i / 3][j / 3][digit] = true;
                    board[i][j] = digit + '0' + 1;
                    dfs(board, pos + 1); //递归下个空白格（Pos+1）
                    line[i][digit] = column[j][digit] = block[i / 3][j / 3][digit] = false;
                }
            }
        }

        void solveSudoku(vector<vector<char>>& board) {
            memset(line, false, sizeof(line));
            memset(column, false, sizeof(column));
            memset(block, false, sizeof(block));
            valid = false;
            for (int i = 0; i < 9; ++i) {
                for (int j = 0; j < 9; ++j) {
                    if (board[i][j] == '.') {
                        spaces.emplace_back(i, j);
                    }
                    else {
                        int digit = board[i][j] - '0' - 1;
                        line[i][digit] = column[j][digit] = block[i / 3][j / 3][digit] = true;
                    }
                }
            }
            dfs(board, 0);
        }
};
```

题目 2（广度优先法）

一个人在一个由方格组成的矩形区域中按上、下、左、右四个方向移动（每

次移动记为一步），区域内可能存在障碍物，每个障碍物占用一个方格，无法通行。计算人从起点方格移动终点方格所需的最少步数。

输入要求：

测试数据的第一行包含 7 个整数 n、 m、k、x0、y0、x1、y1，分别表示矩形区域的行、列方格数（n 和 m），区域内障碍物的数量（k），人的起点方格坐标（行列号，x0 和 y0），终点方格坐标（行列号，x1 和 y1）。之后的 k 行，每行包含两个整数，分别表示障碍物所占用的方格的行列号。

输出要求：

输出 1 行。如果能够到达为所需的最少步数，否则输出 Impossible。

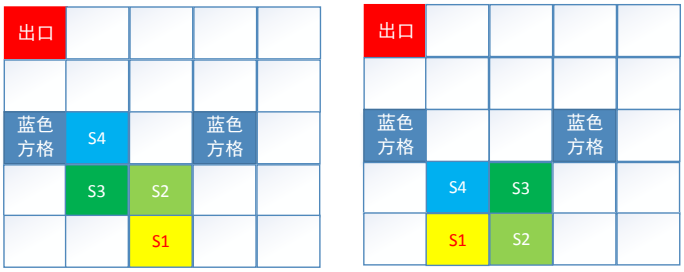
要求：

1、采用标准的广度优先算法编写程序实现上述题目要求。

答案：略。

题目 3（分支限界法）：

可能有不少的同学玩过“贪吃蛇“的游戏，游戏中蛇头带动整个蛇的移动，蛇身将沿蛇头移动过的位置进行移动。如果将整条蛇分成若干个方格，则可以表示为 S1,S2,S3，…，SL，其中 S1 为蛇头，SL 为蛇尾，中间则是蛇头到蛇尾之间的部分。蛇在某个区域内移动时，如果蛇头所在位置的上下左右四个方向没有其它的物体，则蛇头 S1 可以朝其中任何一个方格移动，蛇身则填补前面移动后的区域，也就是 S2 移动到 S1 所在区域，S3 移动到 S2 所在区域……，依次类推，如下图所示。



备注：左图为蛇移动前的情况。右图为蛇向下移动一个方格后的情况。蓝色填充的方格表示有障碍物。

请判断蛇是否可以从它当前所在的位置移动到出口所在的位置，如果能，则给出最少需要移动的步数。如果不能，则输出“Impossible”。

输入要求：

输入的第 1 行包含三个整数 n, m（l<=n, m<=20） 和 L （2<=L<=8），分别表示蛇所在区域的大小（行，列方格数）以及蛇的长度。其后的 L 行用于表示从蛇头到蛇尾依次占用的方格的行列值。紧接着的 1 行包含一个整数 K，表示该区域内障碍物占用的方格数。之后

的 K 行，每行包含两个整数，分别表示障碍物所占用的方格的行列值。出口所在的位置为（1，1），且出口处没有障碍物。

输出要求：

输出 1 行，如果蛇能够到达出口，则输出蛇从当前位置移动到出口位置最少需要移动的方格数量，否则输出“Impossible”。

样例输入：

5 5 4

5 3

4 3

4 2

3 2

2

3 1

3 4

要求：

1、写出问题的目标函数、约束函数和限界函数。手工计算样例的输出值。

答：

目标函数：x==1 && y==1 && dist==mindist（计算到出口的最少步数）

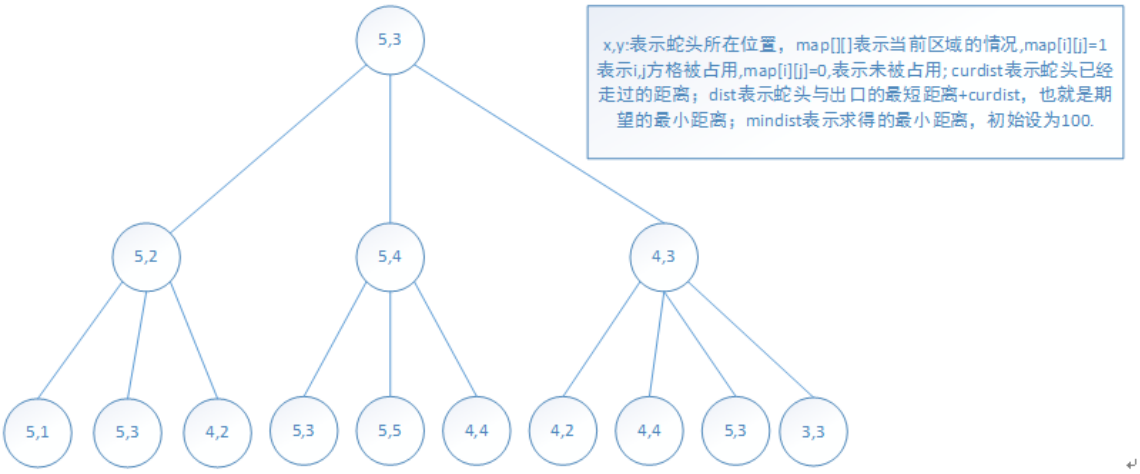
约束条件：x>=1 && x<=N && y>=1 && y<=M && map[i][j]=0

限界函数： dist=curdist+(x-1)+(y-1) （曼哈顿距离）

样例输出：8

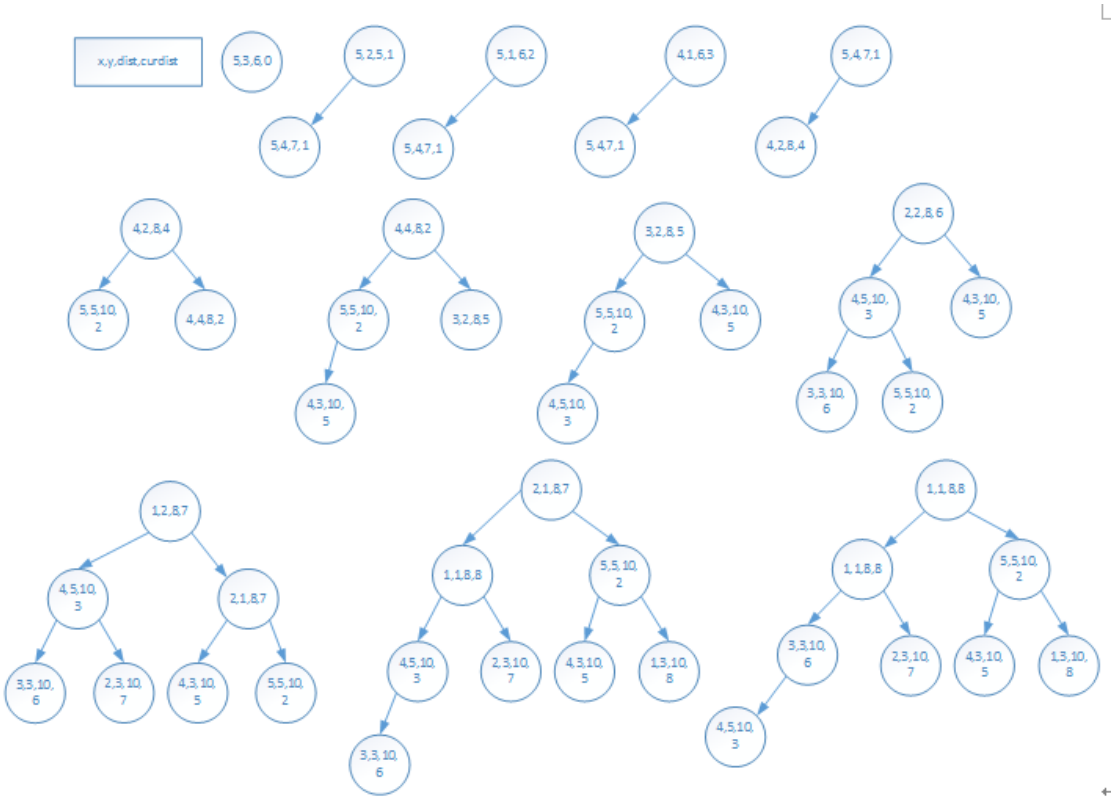
时间复杂性：4^{m*n}（m 和 n 是棋盘的行和列数）

2、画出样例输入时的解空间树（只画两层，在树的节点中标出蛇头坐标）。



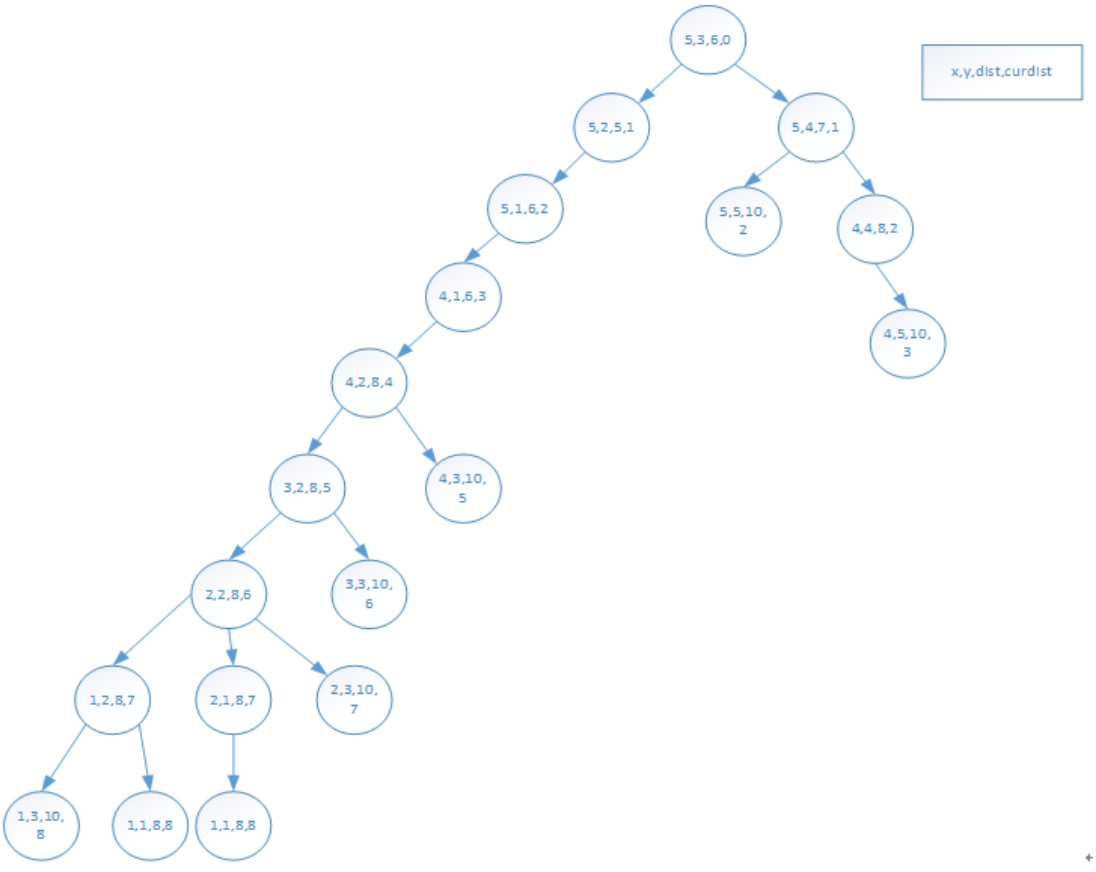
3、画出用分支限界法求解该问题时堆的变化过程，指明堆中每个结点的值及其含义。

答：



4、画出采用分支限界法求解该问题时的搜索空间树。

答：搜索空间树



5、源代码

略

题目 4（分支限界法）

小李希望开车到全国各地旅游，他发现旅途中各个城市的汽油价格也不相同，显然如果能够采取合理的加油方式，将会节省整个旅途的费用。假设汽车开始时油箱为空，汽车每走一个单位的距离将耗费一个单位的汽油。请帮助小李找到一种最省钱的方式完成他的整个自驾游。

输入要求：

输入的第一行包含两个整数 n （1 ≤ n ≤ 1000）和 m（0 ≤ m ≤ 10000），分别表示城市的数量以及道路的数量。其后的一行包含 n 个整数，分别表示 n 个城市的汽油价格。其后的 m 行，每一行包含三个整数 u,v,d，表示城市 u，v 之间的距离为 d（1 ≤ d ≤ 100）。最后一行包含三个整数 c（ 1 ≤ c ≤ 100）,s,e,分别表示油箱的容量，出发的城市和最后到达的城市。

输出要求：

如果能够帮助小李找到一个最省钱从 s 到 e 的路线，则输出对应的费用，否则输出“impossible”。

样例输入：

4 6

8 10 6 7

0 1 8

0 3 12

1 2 10

2 3 7

1 3 9

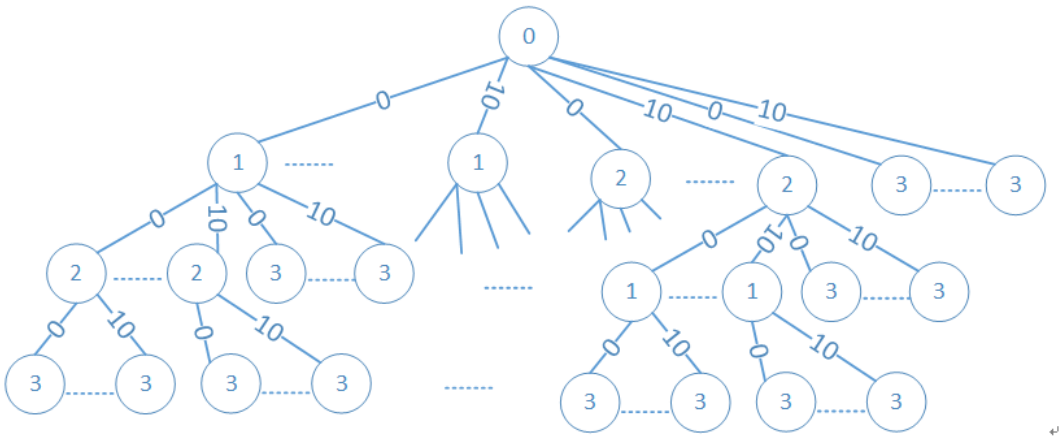
0 2 10

10 0 3

要求：

1、画出采用分支限界法求解样例输入时的解空间树（必要时可以用省略号）。

答：



其中：线条上的数字表示加油的数量，圆中的数字表示城市的编号。

2、给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数。

答：

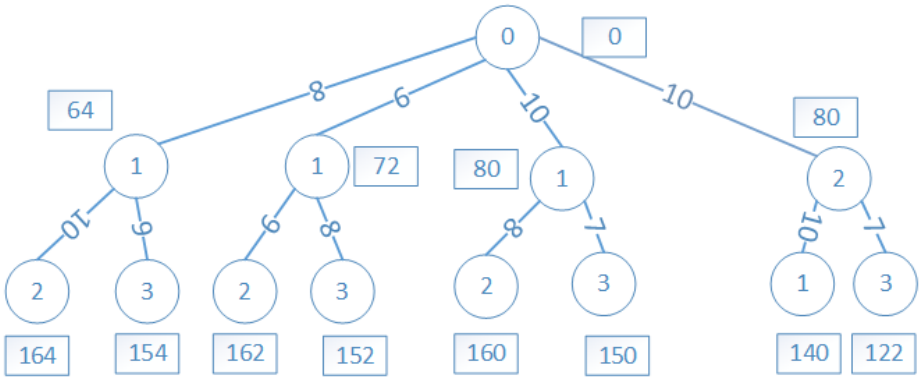
目标函数：到达终点 e 并且总费用最低

约束条件：(加油量<=油箱容量) and (油量>=城市的距离)

限界函数： 当前的总花费

样例输出：122

样例的搜索空间树：



其中：线条上的数字表示加油的数量，圆中的数字表示城市的编号,方框中的数字表示费用。

另一个测试用例：

4 5

10 10 40 20

0 1 4

0 2 3

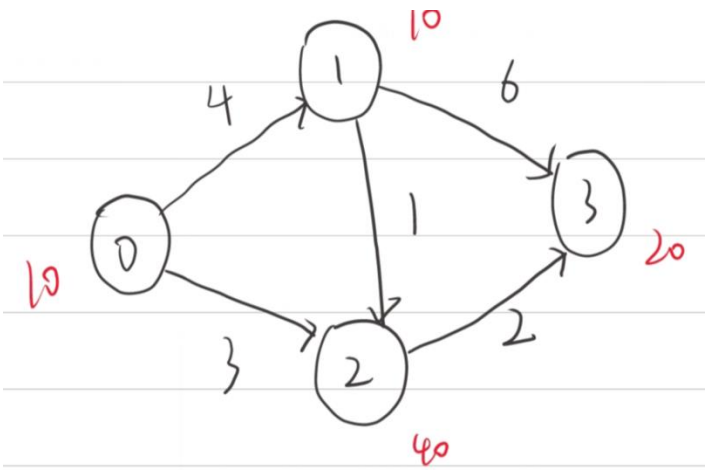
1 2 1

1 3 6

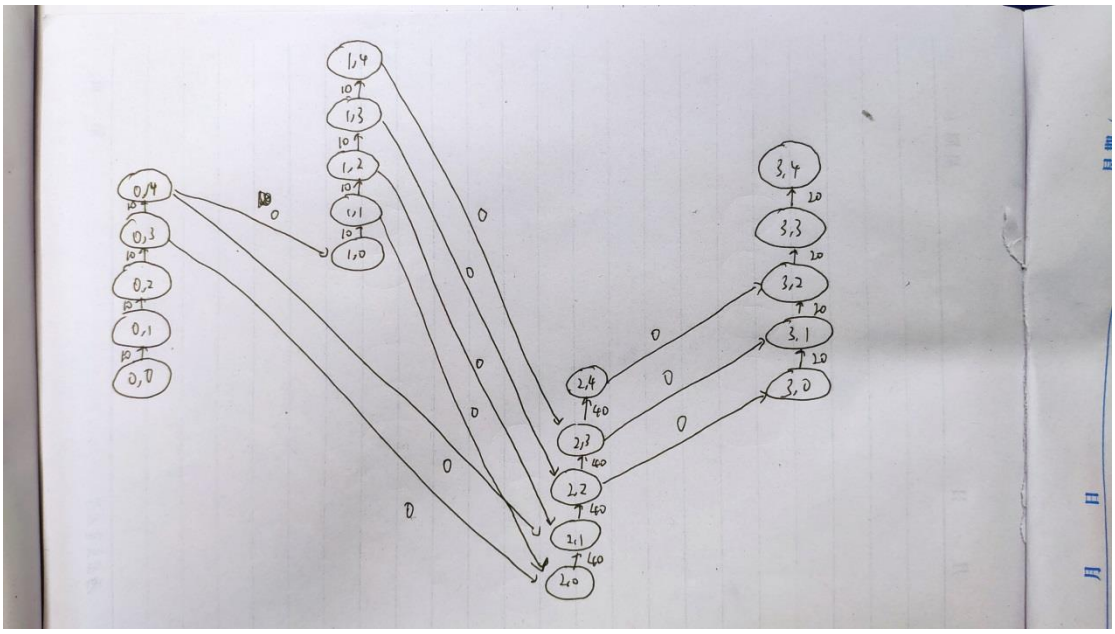
2 3 2

4 0 3

该用例的原图



对该用例原图扩展后的图：



参考代码

```
const int maxn = 5e6;
const int inf = 0x3f3f3f3f;
int head[maxn], vex[maxn], edge[maxn], nxt[maxn], idx;
int dist[maxn];
bool vis[maxn];
inline void addedge(int x, int y, int w){
    idx++;
    vex[idx] = y;
    edge[idx] = w;
    nxt[idx] = head[x];
    head[x] = idx;
```

```
}
void dij(int snode){
	memset(dist, 0x3f, sizeof(dist));
	memset(vis, 0, sizeof(vis));
	priority_queue<pair<int, int>, vector<pair<int,int>>, greater<pair<int,int>>> heap;//dist,
vex;
	dist[snode] = 0;
	heap.push(make_pair(0, snode));
	while(!heap.empty()){
		auto it = heap.top();
		heap.pop();
		int distance = it.first;
		int curnode = it.second;
		if(vis[curnode]) continue;
		vis[curnode] = true;
		for(int i = head[curnode]; i; i = nxt[i]){
			int adjv = vex[i];
			if(vis[adjv]) continue;
			int temp = distance + edge[i];
			if(temp < dist[adjv]){
				dist[adjv] = temp;
				heap.push(make_pair(temp, adjv));
			}
		}
	}
}
int run(){
	int n, m;
	cin >> n >> m;
	vector<int> price(n + 1);
	for(int i = 1; i <= n; i++){
		cin >> price[i];
	}
	vector<tuple<int, int, int>> arc(m + 1);
	for(int i = 1; i <= m; i++){
		int u, v, d;
		cin >> u >> v >> d;
		u++;
		v++;
		arc[i] = make_tuple(u, v, d);
	}
	int c, s, e;
	cin >> c >> s >> e;
	s++;
```

```
    e++;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= c; j++){
            addedge(i + (j - 1) * n, i + j * n, price[i]);
        }
    }
    for(int i = 1; i <= m; i++){
        auto[u, v, d] = arc[i];
        for(int j = d; j <= c; j++){
            addedge(u + j * n, v + (j - d) * n, 0);
        }
    }
    dij(s);
    if(dist[e] < inf) cout << dist[e] << "\n";
    else cout << "impossible\n";
    return 0;
}
```